Lecture # 20

Date: Nov 6th, 25 Day: Thursday

BFS (Queue)

for each node of graph:
- → color
- → distance (d)
- → (π) predecessor

if colon:
(white → unvisited vertex)
gray → partially visited (discovered)
black → completely visited (discovered)

BFS (G, Src) {

initialization: for each vertex "v" ∈ G.V
v.color = white
v.d = ∞
v.π = NULL

src.d = 0
src.color = grey

Q = Queue()
Q.Enqueue (src)

traversal: while(!Q.Empty()) {

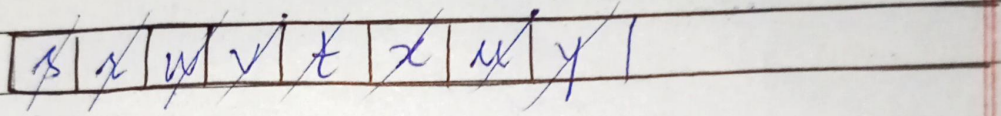
u = Q.dequeue() // extract

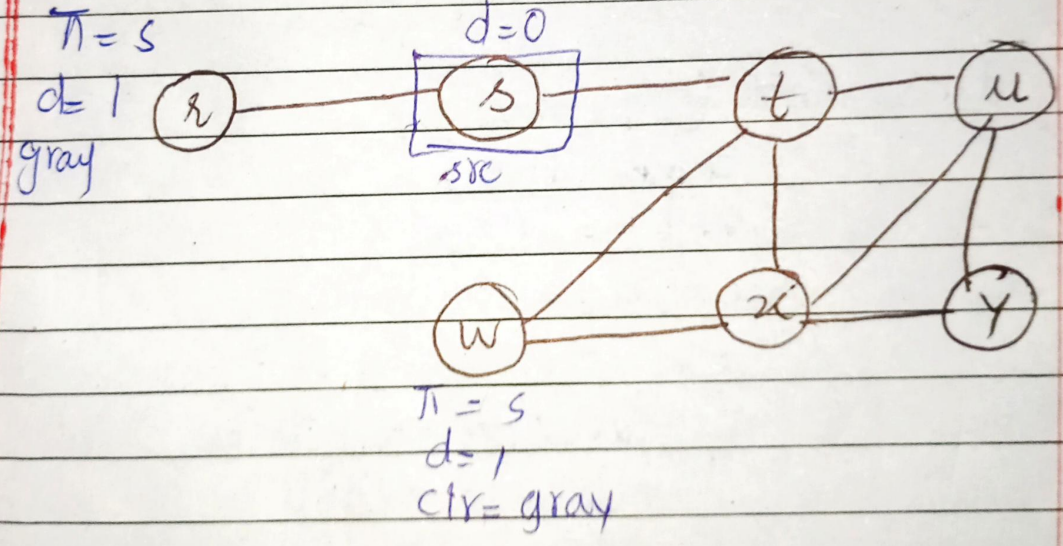
adjacent of the node: for each vertex "v" ∈ G.Adj[u] {
Q.Enqueue (v)

Date:

```
v.π = u
v.d = u.d + 1;
v.color = grey
}
u.color = black;
}
```

predecessor
ki cost +1





Time Complexity :

```
if (v.color == white)
    Q.Enqueue (v)
    v.d = u.d + 1 ;
    v.π = u .....
```

Lecture #21

Date: 11/11/24 Day: Tuesday

DFS

color.
predecessor (π)
discovery time (d)
finish time (f)

```
DFS(G){
    for each 'v' ∈ G.V
        v.color = white
        v.π = null
    for each v ∈ G.V
        if (v.color == white)
            DFS_visit(G, v)
}

time = 0                    // Global variable

DFS_visit(G, src){
        time++;
    src.d = time
    src.color = gray
    for each v ∈ G.adj[src]{
        if (v.color == white){
            v.π = src
            DFS_visit(G, v)
        }
    }
    time++
    src.f = time
    src.color = black
}
```

Date: Day:

Classification of Edges:

1. Tree Edge:
   Edge connecting vertex to an unvisited vertex.

2. Back Edge:
   Edge connecting vertex to it's predecessor's (partially visited vertex)
   → Existence of Back edge indicates a cyclic graph.
   → Remove Backedges → acyclic edge

3. Forward Edge:
   Edge going to a vertex which could have been discovered earlier if different path was used.

4. Cross Edge:

Date: Day:

```
time = 0
DFS-visit (G, src){
    time++
    src.d = time
    src.color = gray
for each v ∈ G.V
  {
      if(v.color == white) {
            V.PI = src
            DFS-visit (G, v)
      }
    elseif (v-color == gray)
        (src, v) is a back edge
   else if (src.d < v.d)
        (src, v) is a Forward edge
   else if (src.d > v.d)
              is a Cross edge

  }
        }
```

back ↑ detectes

DFS Application

Lecture # 22

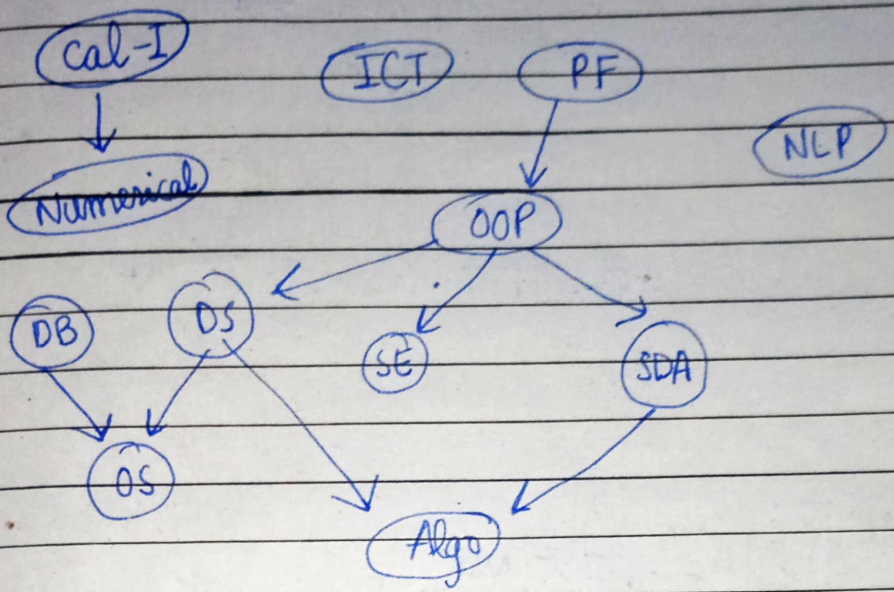
Represents a graph in a linear way.

Date: Nov 13, 25

Day: Thursday

Topological Sort

Pre Condition: (Graph must be Acyclic)




```
TopologicalSort(G){

    for each v ∈ G.V
        v.depCount = 0          // initialization

    for each v ∈ G.V
        for each u ∈ G.Adj[v]
            u.depCount++

    ReadyList, Sol-list = { }

    for each v ∈ G.V
        if v.depCount == 0
            readyList.append(v)
```

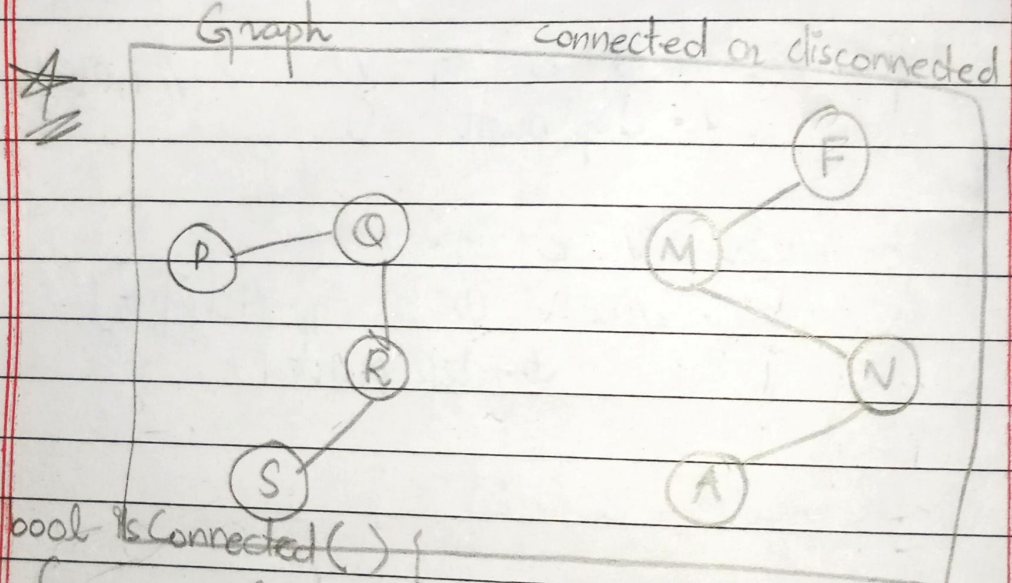
Date: Day:

```
while( readyList != { } ){
    v = ReadyList.removeHead()
    for each u ∈ G.Adj[v] {
        u.depCount -= 1
        if ( u.depCount == 0)
            readyList.addToTail(u)
    }
    sol_List.insertAtTail(v);
}
```

readyList :

PF	CalI	ICT	NLP	DB

d.count == 0



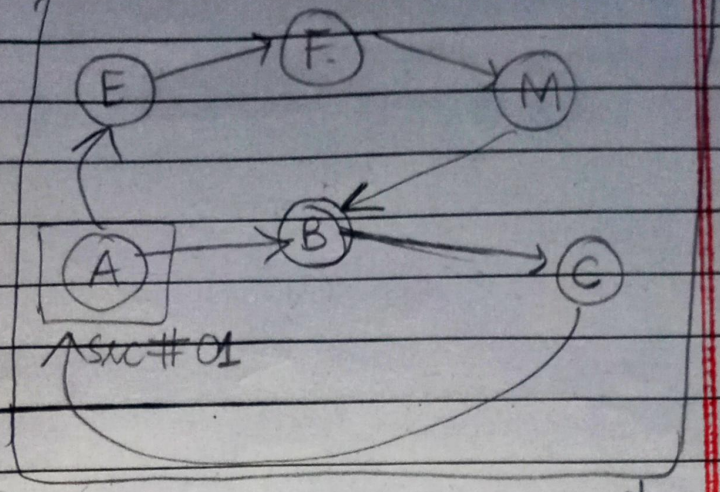

```
bool IsConnected() {
    for any v ∈ G.V
        DFS_visit(G, v)
    for each v ∈ G.V
        if (v.color == white)
            // disconnected graph
            return false;
        else
            return true;
}
```

vertex and ... of the graph .....

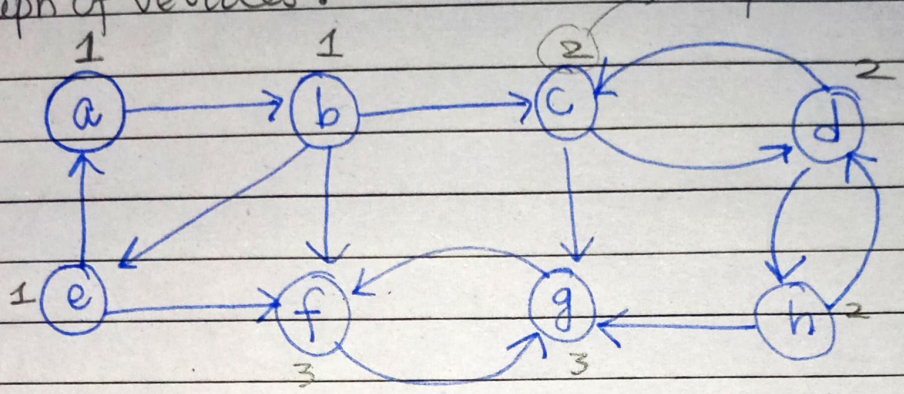
Date: Day:

Graph :-

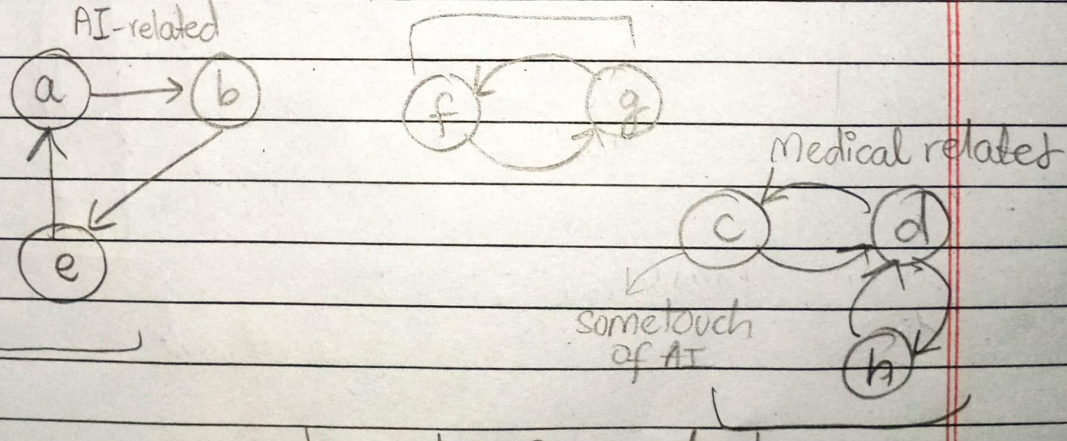
"in the case of directed graph: strongly / weakly connected



Graph of vertices:



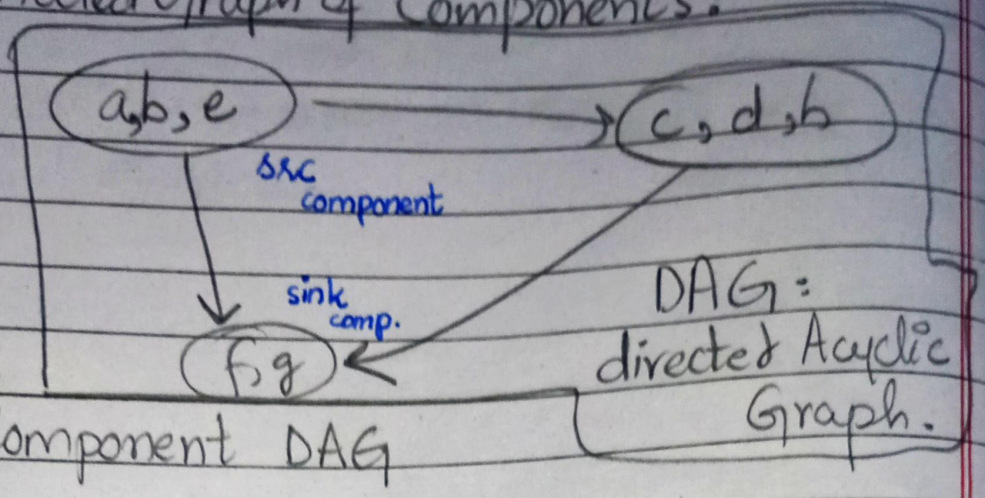
strongly connected component : if we start from one vertex and have a path to return on it.....



stoongly Connected components

Date: arrows ⁰, no outgoing arrow Day:

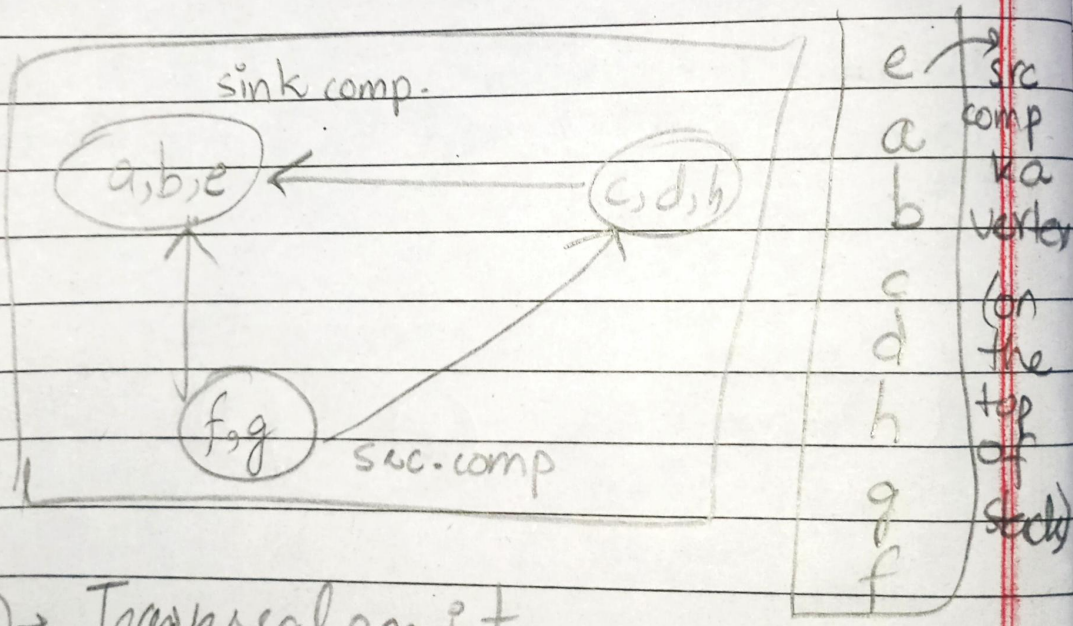
directed Graph of Components:



* start from sink component to assign the component number...

① → DFS ② (maintained Stack.)

③ → Transpose of the Orig Graph.



④ → Traversal on it using the stack